



电子科技大学

University of Electronic Science and Technology of China

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

HTTP 协议概述

Linux下HTTP Server的设计与实现

实训目标

理解TCP与HTTP的分工，掌握请求/响应模式。

能判断HTTP请求是否完整，并解析请求行、请求头和请求体。

能构造200、404等规范响应，并正确设置关键响应头。

能实现访问日志与系统日志，并使用浏览器和curl完成验收。



Epoll管理连接，HTTP解释数据

W2D5: epoll负责发现哪个client_fd发生事件。

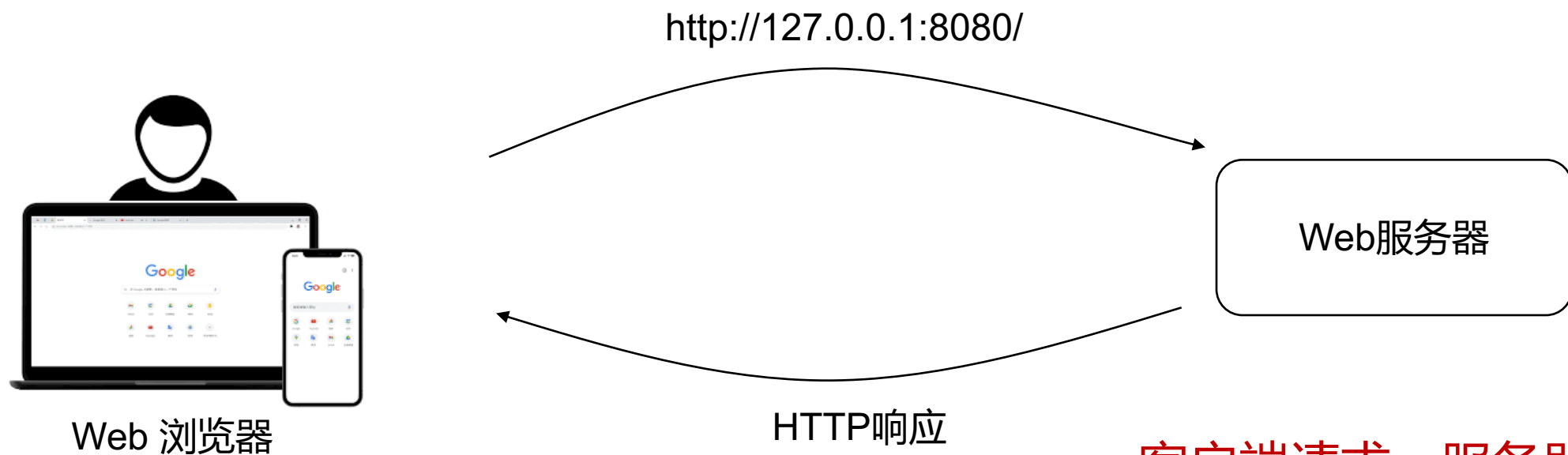
W3D1: HTTP负责解释client_fd中的字节并生成规范响应。

epoll发现事件 → HTTP解析数据 → 路由决定响应 → 日志记录结果



一次HTTP通信包含请求和响应

浏览器访问本地HTTP服务器



客户端请求，服务器响应

1. 浏览器建立TCP连接并发送HTTP请求
2. 服务器解析请求并返回HTTP响应
3. 浏览器解析响应体并渲染页面



HTTP规定应用层数据如何组织和解释

HTTP是应用层协议，采用客户端请求、服务器响应的通信模式。

浏览器通常是HTTP客户端，WebServer是HTTP服务器。

HTTP本身无状态：不同请求原则上相互独立。

无状态不等于每次都断开连接；HTTP/1.1支持持久连接。

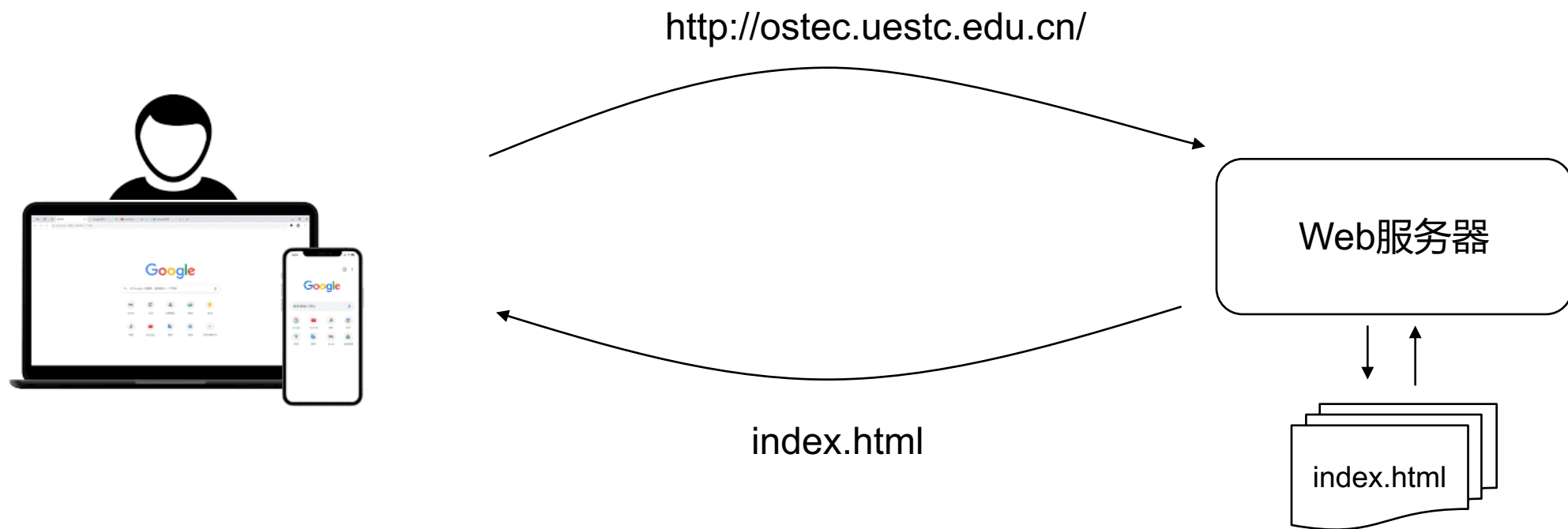
本实训使用HTTP/1.1 over TCP，HTTP默认端口为80。

TCP解决可靠传输，HTTP定义请求和响应的格式与含义。



静态页面直接读取已有资源

用户访问静态网页

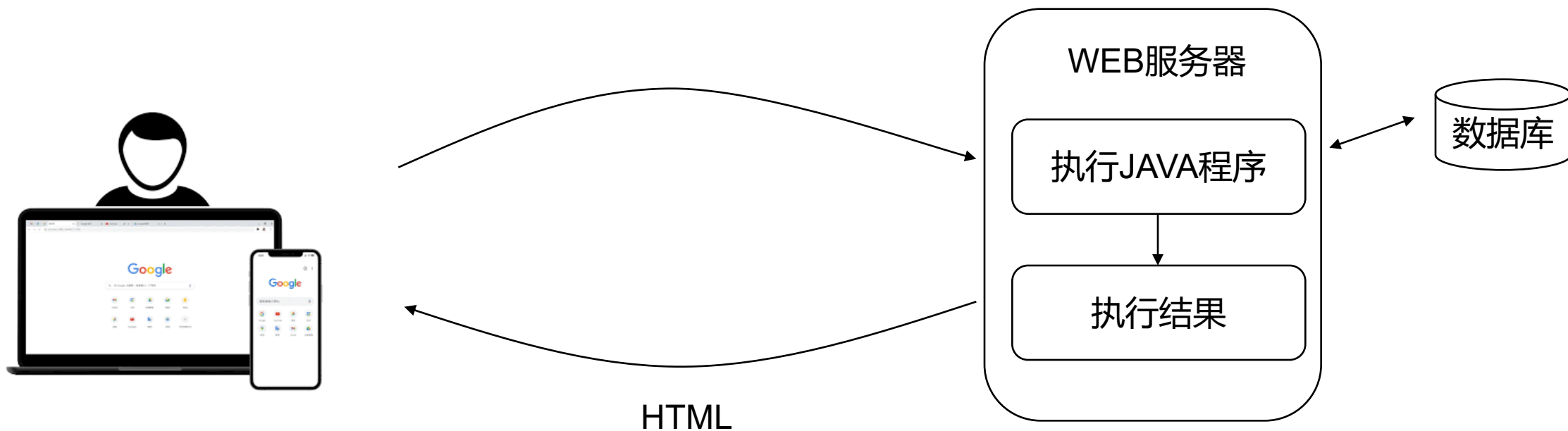


静态页面：服务器直接返回已经存在的 HTML、CSS、图片等文件。



动态页面由程序按请求生成

用户访问动态网页



动态页面：服务器根据请求参数、业务逻辑或数据库数据，动态生成响应内容。



HTTP 协议概述



HTTP通信使用的是TCP还是UDP呢?

OSI 七层网络模型与TCP/IP模型

@稀土掘金技术社区



电子科技大学
University of Electronic Science and Technology of China

信息与软件工程学院
SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

不同HTTP版本使用不同传输方式

本次实训：HTTP/1.1运行在可靠的TCP连接之上。

HTTP/1.1和HTTP/2通常基于TCP实现。

HTTP/3基于QUIC，底层使用UDP。

本次程序仍通过socket、accept、recv和send处理HTTP。



EpollServer处理HTTP请求的完整流程

socket → bind → listen → 将listen_fd注册到epoll

epoll_wait → accept或recv → 判断请求是否完整

HTTP解析 → 路由处理 → 构造响应 → send

记录日志 → epoll_ctl(DEL) → close(client_fd)



URL由协议、主机、端口和路径组成

URL用于定位服务器上的网络资源

协议：告诉服务器使用什么通信协议（这里是HTTP）

端口号：一个服务器应用应该由一个端口标识，默认：端口 80

资源：所请求内容的名字，可以是一个HTML页面、图片、音乐、视频等

`http://www.example.com:80/image/123.jpg`

域名：最终会映射到一个IP地址

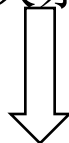
路径：所请求资源在其服务上的路径



路径映射由服务器路由规则决定

浏览器发送的是请求目标，服务器决定它对应哪个资源

访问 `http://www.example.com/` 时，请求目标是 `/`。
服务器可以将 `/` 映射到 `index.html`，这属于服务器路由规则。



显式访问：`http://www.example.com/index.html`

一次recv()不等于一个完整HTTP请求

TCP只提供字节流，请求可能分多次到达，也可能与后续数据一起到达。

1. 将recv()收到的字节追加到该客户端的接收缓冲区。
2. 查找\r\n\r\n，判断请求头是否完整。
3. 解析Content-Length；存在请求体时继续接收指定字节数。
4. 请求完整后再解析请求行、请求头和请求体。

recv → 追加缓冲区 → 判断完整 → HTTP解析



HTTP 请求与响应

HTTP 协议是一个请求-响应式协议，一般由客户端（比如浏览器）向服务端发起，服务端处理后，再向客户端返回信息。

客户端发出的消息称为HTTP 请求（Request），服务端返回的消息称为HTTP 响应（Response）。



每种协议都有它的消息结构，有了消息结构，发送方就知道以怎样的结构发送消息，接收方就知道以怎样的结构接收消息。



HTTP 请求消息结构



HTTP 请求由四部分组成:

- ① 请求行: 分别是请求方法+空格+URL+空格+协议版本+\r\n
- ② 请求头部: 由多个请求头部键值对组成, 中间以冒号:隔开, 每个键值对最后是\r\n
- ③ 空行: 即\r\n
- ④ 请求包体: 包体部分

HTTP例子

GET / HTTP/1.0

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_5)

Accept: */*

- 第1行是请求行，GET 为请求方法，/ 为URL，HTTP1.0 为协议版本，中间以空格隔开。
- 第2行与第3行是请求头。
- 请求头下面的空行，这里没有显示。
- 一般GET 请求不包含请求体，所以本请求中没有请求体。



HTTP 请求响应结构



HTTP 响应由四部分组成:

状态行: 分别是协议版本+空格+状态码+空格+状态码描述+\r\n

响应头部: 由多个响应头部键值对组成, 中间以冒号:隔开, 每个键值对最后是\r\n

空行: 即\r\n

响应包体: 包体部分



HTTP 请求响应例子

HTTP/1.0 200 OK

Content-Type: text/plain

Content-Length: 82

Expires: Thu, 05 Dec 1997 16:00:00 GMT

Last-Modified: Wed, 5 August 1996
15:55:28 GMT

Server: Apache 0.84

<html>

<body>Hello HTTP</body>

</html>

- 第1行为响应状态行，HTTP/1.0 为协议版本，200 是响应状态码，OK是状态码描述，中间用空格隔开。
- 第2行到第6行为响应头。
- 第7行为空行。
- 第8行到第10行为响应内容。



本次实训采用的最小HTTP响应格式

HTTP/1.1 200 OK\r\n

Content-Type: text/html; charset=utf-8\r\n

Content-Length: 21\r\n

Connection: close\r\n

\r\n

<h1>Hello, HTTP!</h1>

每行以 \r\n 结束；空行分隔响应头和响应体；Content-Length是响应体的字节数。



本次实训重点掌握GET和POST

HTTP 协议支持9 种请求方法，最常用的是GET 和POST 方法。

请求方法	含义
<i>GET</i>	获取指定页面信息
<i>HEAD</i>	类似 <i>GET</i> 请求，但响应中没有具体内容，只有响应 <i>Header</i>
<i>POST</i>	向指定 <i>URL</i> 提交信息
<i>PUT</i>	向服务器传送数据
<i>DELETE</i>	请求服务器删除指定内容
<i>OPTIONS</i>	查看服务器信息
<i>TRACE</i>	回显服务器收到的请求，主要用于测试或诊断
<i>PATCH</i>	是对 <i>PUT</i> 方法的补充，用来对已知资源进行局部更新
<i>CONNECT</i>	<i>HTTP/1.1</i> 协议中预留给能够将连接改为管道方式的代理服务器

必做范围：GET /、GET /missing、POST /echo



HTTP GET 与POST 方法

GET 方法与POST 方法是最常用的两个HTTP 方法，来看下其异同点：

- 请求内容存放位置不同：GET 方法一般没有请求体，其请求内容放在URL 参数中，POST 方法则将请求内容放在请求体中。
- POST 方法 安全性更高：GET 请求一般是明文传输，不利于传输敏感数据。POST 请求内容在请求体中，更方便加密，提高安全性。
- POST 方法传输的数据量更大：GET 请求内容在URL 中，因此有大小限制，而POST 请求 内容在请求体中，理论上没有大小限制。



HTTP 请求头字段

Host: 客户端请求的域名。

Connection: 告诉服务端，处理完本请求后，是否关闭连接。

User-Agent: 客户端使用的浏览器或APP 类型/版本。

Accept: 客户端支持哪些类型的文档。

Accept-Encoding : 客户端支持的编码类型。

Accept-Language : 客户端支持的语言类型。

Referer : 客户端从哪个网页过来的。

Cache-Control: 指定缓存机制。



HTTP 响应头字段

Allow：表明服务器支持哪些请求方法，如GET，POST 等。

Content-Encoding：响应内容编码方法。

Content-Type：响应内容属于什么MIME 类型。

Content-Length：响应内容的长度。

Date：当前GMT 时间。

Expires：响应内容过期时间，过期后将不再缓存内容。

Last-Modified：文档的最后改动时间。

Location：告诉客户端到哪里获取文档，一般用于重定向。

Refresh：浏览器在多少秒后刷新文档。

Server：服务器名字。

Set-Cookie：设置和页面关联的Cookie。

Date：表示消息发送时间。



状态码说明服务器的处理结果

1xx 信息：请求已收到，继续处理。

2xx 成功：请求已经正确处理，例如200 OK。

3xx 重定向：客户端需要访问其他位置。

4xx 客户端错误：请求无效或资源不存在，例如400、404、405。

5xx 服务器错误：服务器处理请求时发生错误，例如500。

本次实训重点：200、404、405。



浏览器和curl用于观察真实HTTP通信

浏览器Network适合观察页面加载过程和资源请求。

curl适合进行稳定、可重复的命令行测试。

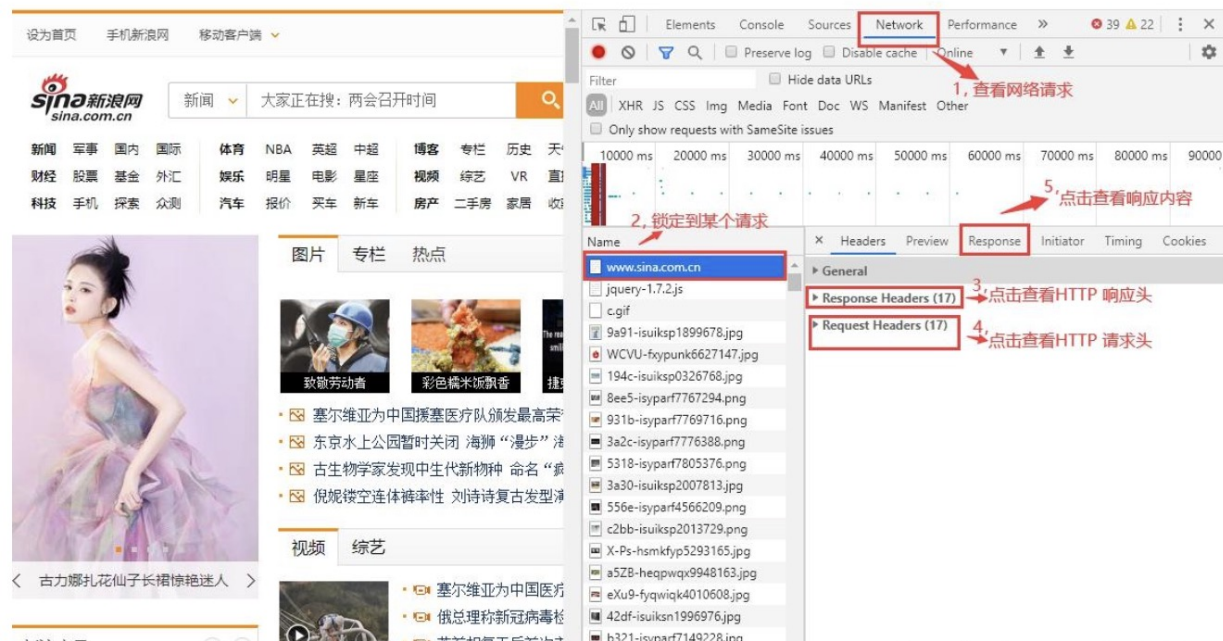
需要重点观察：

- Request Method和Request URL
- Request Headers与请求体
- Status Code和Response Headers
- Response Body与响应时间



用浏览器演示HTTP

在浏览器（我这里用的是Chrome）中访问新浪网https://www.sina.com.cn/，通过F12键，调出浏览器开发者工具。



1. 点击Network，可查看网络请求
2. 点击某个请求，可查看该请求信息
3. 点击Response Header，可查看响应头内容
4. 点击Request Header，可查看请求头信息
5. 点击点击Response，可查看响应内容



使用curl完成可重复的HTTP测试

正常路由: `curl -i http://127.0.0.1:8080/`

404路由: `curl -i http://127.0.0.1:8080/missing`

POST请求:

```
curl -i -X POST -H "Content-Type: text/plain" \  
-d "hello HTTP" http://127.0.0.1:8080/echo
```

-i用于同时显示响应头和响应体; -v用于观察完整通信过程。

日志让服务器行为可追踪、可定位

当服务器同时处理多个客户端时，仅观察终端输出很难定位问题。

系统日志：记录服务器启动、退出、警告、系统事件和错误。

访问日志：记录客户端访问资源的过程和处理结果。

两类日志必须写入不同文件，便于运行监控、故障排查和访问分析。



系统日志记录运行状态和错误

示例:

2026-07-18 09:00:00 [INFO] server started, listen=127.0.0.1:8080

2026-07-18 09:02:13 [ERROR] recv failed, fd=7, errno=54

最低字段：时间、日志级别、事件描述。

典型事件：启动、退出、accept失败、recv失败、请求解析失败、文件读取失败。



访问日志记录一次请求的处理结果

示例:

2026-07-18 09:05:21 127.0.0.1 GET / HTTP/1.1

200 21 1.2ms curl/8.7.1

最低字段:

时间、客户端IP、请求方法、URL、HTTP版本、状态码、响应字节数、响应时间、User-Agent。



系统日志按严重程度分级

DEBUG：调试细节，用于开发和深入分析问题。

INFO：正常事件，例如服务器启动和停止。

WARNING：值得关注但暂不影响运行的问题。

ERROR：已经发生并影响本次操作的错误。

CRITICAL：可能导致服务器无法继续运行的严重错误。

必做支持：DEBUG、INFO、WARNING、ERROR。



实训任务：构建HTTP服务器与日志系统

1. 复用W2D5的EpollServer，按客户端维护HTTP接收缓冲区。
2. 解析请求行、请求头和可选请求体。
3. GET /返回200和HTML；GET /missing返回404。
4. POST /echo解析请求体并返回200。
5. 响应正确设置Content-Type、Content-Length和Connection。
6. 响应完成后从epoll删除并关闭client_fd。
7. 系统日志和访问日志分文件记录，系统日志支持四种级别。



最低验收一：HTTP功能与连接清理

- 服务器能够在指定IP和端口正常监听。
- GET /返回200 OK，浏览器能够显示HTML页面。
- GET /missing返回404 Not Found，服务器不崩溃。
- POST /echo能够解析请求体并返回200。
- 响应使用\r\n，Content-Type和Content-Length正确。
- 连续执行三类请求后，其他客户端仍能正常访问。
- 响应完成后执行epoll_ctl(DEL)并close(client_fd)。



最低验收二：日志记录

- 系统日志和访问日志分别写入不同文件。
- 访问日志至少记录时间、客户端IP、方法、URL和状态码。
- 系统日志至少记录时间、级别和事件描述。
- 日志模块支持DEBUG、INFO、WARNING、ERROR。
- 正常请求、404请求和POST请求都有对应访问记录。

两部分全部通过，才能认定达到最低验收要求。



从连接事件到可验收的HTTP服务器

epoll_wait发现事件

- recv追加客户端缓冲区
- 判断HTTP请求完整
- 解析请求行、请求头和请求体
- 路由生成200、404或405响应
- send发送响应
- 记录访问日志和系统日志
- epoll_ctl(DEL)并close客户端fd

TCP负责传输字节；HTTP负责解释字节；日志负责留下处理证据。

